

// C++20 · LEARN IT IN TERMINAL TOWN

C++ Cheat-Sheet

Type it. Break it. Fix it. Ship it.



ROSETTA STONE START HERE

JavaScript to C++

JavaScript	C++
`let x = 5;`	`int x = 5;`
`const y = 5;`	`const int y = 5;`
`console.log(x)`	`std::cout << x;`
`arr.push(1)`	`v.push_back(1);`
`arr.length`	`v.size()`
`for (a of arr)`	`for (int a : v)`
`(x) => x * 2`	`[](int x){...}`
`{ hp: 5 }`	`struct S { int hp; };`
`null`	`nullptr`
`x === y`	`x == y`

THE LOOP

Build and run

```cpp
#include <iostream>
int main() {
  std::cout << "Hi, Orby!\n";
  return 0;
}
```

cl /std:c++20 /W4 /EHsc main.cpp

TYPES

Every value has a type

```cpp
int hp = 100;
double speed = 2.5;
bool alive = true;
std::string name = "Orby";
const int MAX_HP = 100;
auto xp = 42;
```

C++ checks types BEFORE you run.

FLOW

Decide and loop

```cpp
if (hp <= 0) {
  alive = false;
} else if (hp < 20) {
  std::cout << "Low!\n";
}
while (alive) { /* a turn */ }
for (int i = 0; i < 3; ++i) { }
```

ENUM + SWITCH

Names for choices

```cpp
enum class State { Idle, Fight, Dead };
State s = State::Fight;
switch (s) {
  case State::Idle: break;
  case State::Fight: Attack(); break;
  default: break;
}
```

FUNCTIONS

Functions

```cpp
int RollDice(int sides) {
  return 1 + rand() % sides;
}
void Say(std::string m); // declare
```

STRING

Text

```cpp
std::string s = "Hello";
s += ", Orby";
s.size();          // 11
s.substr(0, 5);    // "Hello"
std::string line;
std::getline(std::cin, line);
```

VECTOR

Lists that know their type

```cpp
std::vector<int> bag = {5, 8};
bag.push_back(13);
bag.size();        // 3
bag[0];            // 5
for (int item : bag) { }
```

MAP

Look things up by name

```cpp
std::map<std::string, int> loot;
loot["gold"] = 50;
if (loot.count("gold")) { }
for (auto& [k, v] : loot) { }
```

STRUCT

Your own types

```cpp
struct Fighter {
  std::string name;
  int hp = 10;
};
Fighter g{"Goblin", 8};
g.hp -= 3;
```

REFERENCES MUST MEMORIZE

Copy or original?

```cpp
void Heal(Fighter& f) {
  f.hp += 5;  // the original
}
void Show(const Fighter& f);
  // read-only, no copy
void Oops(Fighter f) { }
  // a COPY!
```

JS shares objects. C++ COPIES!

CLASS

Classes guard their data

```cpp
class Hero {
public:
  Hero(int hp) : hp_(hp) {}
  int Hp() const { return hp_; }
private:
  int hp_;
};
```

POINTERS

Pointers, honestly

```cpp
Fighter* t = &g;  // address of g
t->hp = 5;        // (*t).hp
Fighter* none = nullptr;
if (t != nullptr) { }
```

JS: every object var is secretly one

Never use a pointer after its object is gone.

HEADERS

Many files

```cpp
// Hero.h
#pragma once
class Hero {
public: int Hp() const; };

// Hero.cpp
#include "Hero.h"
int Hero::Hp() const { return 7; }
```

.h declares. .cpp defines.

DESTRUCTOR (RAII)

Cleanup runs by itself

```cpp
class Log {
public:
  Log()  { std::cout << "open\n"; }
  ~Log() { std::cout << "close\n"; }
};
{ Log l; }  // ~Log() runs at }
```

VIRTUAL

One interface, many monsters

```cpp
struct Monster {
  virtual ~Monster() = default;
  virtual void Attack() = 0;
};
struct Goblin : Monster {
  void Attack() override { }
};
```

UNIQUE_PTR

Who owns this?

```cpp
auto m =
  std::make_unique<Goblin>();
m->Attack();
std::vector<
  std::unique_ptr<Monster>> horde;
// freed automatically. No delete!
```

JS: the garbage collector does it

One owner. Cleanup is automatic.

LAMBDAS

Tiny functions on the spot

```cpp
auto dbl = [](int x) { return x * 2; };
dbl(4);               // 8
int bonus = 3;
auto add = [bonus](int x) {
  return x + bonus;   // captured
};
std::function<void(int)> onHit;
```

*JS: (x) ⇒ x * 2*



Boss fights: common bugs



Compiler yelling? Read the FIRST error only. Fix it. Rebuild.
Copy trap No & means a copy. Use Fighter& to change the original.
Missing semicolon A class or struct ends with }; (the semicolon!)
Garbage value int hp; has no value. Write int hp = 0;
Dangling pointer Never use a pointer after its object is gone.
Off by one A vector of 3 has indexes 0, 1, 2. Not 3.